

PRÁCTICA FINAL DE UNIDAD I

Estructuras de Datos y Algoritmos

Caso Integrador: Mini Hoja de Cálculo Académica

1. Título de la Práctica

Diseño e implementación de una mini hoja de cálculo académica utilizando estructuras de datos avanzadas y análisis asintótico de algoritmos en C++ y Python.

2. Objetivo General

Desarrollar una mini hoja de cálculo académica orientada a terminal que permita registrar, consultar, ordenar, modificar y analizar datos de rendimiento estudiantil mediante la convergencia ordenada de listas dinámicas, pilas, colas, tablas hash, recursividad y algoritmos de ordenamiento de tipo "divide y vencerás". El estudiante comprenderá la abstracción e importancia de las estructuras de datos subyacentes en motores de software comercial como Microsoft Excel, Google Sheets o sistemas ERP académicos de gran escala.

3. Competencias a Evaluar

- **Analizar la complejidad temporal y espacial** de algoritmos fundamentales utilizando la notación formal Big-O (O).
- **Aplicar recursividad eficiente** para la resolución de cálculos acumulativos y estadísticas agregadas sin comprometer la integridad del stack de ejecución.
- **Implementar y comparar algoritmos de ordenamiento** internos (Selection, Insertion, QuickSort, MergeSort) midiendo su desempeño bajo diferentes escenarios de datos (mejor, promedio y peor caso).
- **Diseñar estructuras híbridas y sincronizadas** donde coexistan múltiples mecanismos de acceso (listas dinámicas para persistencia secuencial y tablas hash para indexación directa).
- **Garantizar la integridad de estados de memoria** mediante el uso de pilas para el control de reversión de acciones (patrón Deshacer/Undo).
- **Modelar flujos de atención secuenciales** mediante estructuras lineales de tipo cola (FIFO) para simular procesos operativos reales.
- **Sustentar las diferencias de abstracción, performance y gestión de memoria** entre lenguajes compilados de tipado estático (C++) y lenguajes interpretados de tipado dinámico (Python).

4. Caso de Estudio

La Escuela Profesional de Ingeniería de Sistemas requiere una solución ligera inspirada en una hoja de cálculo para gestionar y auditar las calificaciones del periodo académico actual. Cada fila de nuestra matriz representa un registro único de un estudiante, y las columnas modelan las dimensiones clave del esquema educativo:

Código	Nombre Completo	Nota 1	Nota 2	Nota 3	Promedio Final
2024001	Juan Mamani	14.0	16.0	15.0	15.00
2024002	María Quispe	18.0	17.0	19.0	18.00
2024003	Carlos Condori	11.0	13.0	12.0	12.00

Requerimientos Funcionales Mínimos:

1. **Registro de Estudiantes:** Inserción de nuevas filas validando en tiempo real la no duplicidad de las claves primarias (códigos).
2. **Cálculo Automatizado:** Procesamiento inmediato del promedio ponderado o aritmético a partir de las calificaciones parciales introducidas.
3. **Búsqueda Indexada:** Localización instantánea de registros por código único sin recurrir a barridos secuenciales.
4. **Ordenamiento Multicriterio:** Clasificación descendente por promedio (vía QuickSort) y ascendente por código (vía MergeSort).
5. **Subsistema Deshacer (Undo):** Capacidad de revertir modificaciones o mutaciones sobre la tabla para regresar al estado inmediatamente anterior.
6. **Simulador de Atención Académica:** Cola de atención secuencial integrada para resolver peticiones estudiantiles bajo un esquema FIFO.
7. **Estadísticas Avanzadas:** Cálculo de la media de la cohorte mediante acumulaciones puramente recursivas.

5. Estructuras de Datos Utilizadas y Análisis de Complejidad

5.1 Lista Dinámica

Funciona como el contenedor lineal primario para almacenar de forma contigua los registros de la hoja de cálculo. En C++ se abstrae mediante `std::vector` y en Python mediante la estructura `list`. Proporciona un acceso aleatorio indexado en tiempo constante $O(1)$, pero las inserciones intermedias o búsquedas no indexadas requieren un costo lineal $O(n)$.

5.2 Tabla Hash

Actúa como un índice secundario asociativo que mapea la clave única (Código del Estudiante) con la posición física real (índice) donde reside el objeto dentro de la Lista Dinámica. Implementado con `std::unordered_map` en C++ y `dict` en Python. Ofrece búsquedas, inserciones y eliminaciones en tiempo constante promedio $O(1)$.

5.3 Pila (Stack)

Estructura LIFO (Last-In, First-Out) encargada de apilar instantáneas (snapshots) o estados controlados de la hoja de cálculo cada vez que ocurre una mutación estructural. Permite restaurar el estado anterior en tiempo $O(1)$ eliminando el nodo superior de la pila mediante la operación *pop*.

5.4 Cola (Queue)

Estructura FIFO (First-In, First-Out) diseñada para encolar identificadores de alumnos en espera de atención académica o procesamiento de trámites. Las inserciones (*enqueue*) y remociones (*dequeue*) se realizan estrictamente en los extremos opuestos de la estructura garantizando un costo de $O(1)$.

5.5 Algoritmos de Ordenamiento Comparados

El núcleo analítico requiere la implementación y contraste empírico de las siguientes estrategias de ordenación:

Algoritmo	Mejor Caso	Caso Promedio	Peor Caso	Complejidad Espacial	Estabilidad
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$ In-place	Inestable
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$ In-place	Estable
QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ Auxiliar	Inestable
MergeSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$ Auxiliar	Estable

6. Código Fuente Optimizado en C++

Esta implementación de nivel de producción utiliza el paso de parámetros por referencia constante (`const &`) para mitigar copias redundantes en memoria y optimiza los flujos de entrada/salida estándar.

```
#include <iostream>
#include <vector>
#include <unordered_map>
#include <stack>
#include <queue>
#include <iomanip>
#include <string>
#include <utility>

using namespace std;

struct Estudiante {
    string codigo;
    string nombre;
    float nota1;
    float nota2;
    float nota3;
    float promedio;
};

// Estructuras de almacenamiento global
vector<Estudiante> hoja;
unordered_map<string, int> indiceHash;
stack<vector<Estudiante>> historial;
queue<string> colaAtencion;

// Función en línea para cálculo aritmético eficiente
inline float calcularPromedio(float n1, float n2, float n3) {
    return (n1 + n2 + n3) / 3.0f;
}

// Reconstrucción del índice asociativo O(n)
void actualizarHash() {
    indiceHash.clear();
    for (size_t i = 0; i < hoja.size(); ++i) {
        indiceHash[hoja[i].codigo] = static_cast<int>(i);
    }
}

// Resguardo del estado actual previo a mutaciones
void guardarHistorial() {
    historial.push(hoja);
}

void registrarEstudiante() {
    Estudiante e;
    cout << "
--- REGISTRAR NUEVO ESTUDIANTE ---
";
    cout << "Codigo unico: ";
    cin >> e.codigo;

    // Validación de clave primaria duplicada en tiempo O(1)
    if (indiceHash.find(e.codigo) != indiceHash.end()) {
        cout << "[ERROR] El codigo ingresado ya se encuentra registrado.
";
        return;
    }
}
```

```

    cin.ignore();
    cout << "Apellidos y Nombres: ";
    getline(cin, e.nombre);

    cout << "Nota 1: "; cin >> e.nota1;
    cout << "Nota 2: "; cin >> e.nota2;
    cout << "Nota 3: "; cin >> e.nota3;

    e.promedio = calcularPromedio(e.nota1, e.nota2, e.nota3);

    guardarHistorial();
    hoja.push_back(e);
    actualizarHash();

    cout << "[EXIT0] Registro insertado e indexado correctamente.
";
}

void mostrarHoja() {
    if (hoja.empty()) {
        cout << "
[AVISO] La hoja de calculo no contiene registros.
";
        return;
    }
    cout << "
=====
";
    cout << "
                                MINI HOJA DE CALCULO ACADEMICA
";
    cout << "=====
";
    cout << left << setw(12) << "Codigo"
        << setw(25) << "Nombre Completo"
        << setw(8) << "N1"
        << setw(8) << "N2"
        << setw(8) << "N3"
        << setw(10) << "Promedio" << endl;
    cout << "-----
";

    for (const auto &e : hoja) {
        cout << left << setw(12) << e.codigo
            << setw(25) << e.nombre
            << setw(8) << setprecision(1) << fixed << e.nota1
            << setw(8) << e.nota2
            << setw(8) << e.nota3
            << setw(10) << setprecision(2) << e.promedio << endl;
    }
    cout << "=====
";
}

void buscarEstudiante() {
    string codigo;
    cout << "
Ingrese el codigo a buscar en Tabla Hash: ";
    cin >> codigo;

    auto it = indiceHash.find(codigo);
    if (it != indiceHash.end()) {
        int pos = it->second;
        const auto &e = hoja[pos];
        cout << "
[REGISTRO ENCONTRADO - BUSQUEDA 0(1)]
";
        cout << "Estudiante: " << e.nombre << " | Promedio Actual: " << fixed <<

```

```

setprecision(2) << e.promedio << endl;
    } else {
        cout << "[AVISO] Codigo no registrado en la base indexada.
";
    }
}

// QuickSort: Particionamiento por Promedio (Descendente de Mayor a Menor)
int particion(vector<Estudiante> &arr, int bajo, int alto) {
    float pivote = arr[alto].promedio;
    int i = bajo - 1;

    for (int j = bajo; j < alto; j++) {
        if (arr[j].promedio >= pivote) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[alto]);
    return i + 1;
}

void quickSort(vector<Estudiante> &arr, int bajo, int alto) {
    if (bajo < alto) {
        int pi = particion(arr, bajo, alto);
        quickSort(arr, bajo, pi - 1);
        quickSort(arr, pi + 1, alto);
    }
}

// MergeSort: Combinación por Código Alfanumérico (Ascendente)
void merge(vector<Estudiante> &arr, int izq, int medio, int der) {
    int n1 = medio - izq + 1;
    int n2 = der - medio;

    vector<Estudiante> L(n1), R(n2);

    for (int i = 0; i < n1; i++) L[i] = arr[izq + i];
    for (int j = 0; j < n2; j++) R[j] = arr[medio + 1 + j];

    int i = 0, j = 0, k = izq;

    while (i < n1 && j < n2) {
        if (L[i].codigo <= R[j].codigo) {
            arr[k] = L[i]; i++;
        } else {
            arr[k] = R[j]; j++;
        }
        k++;
    }

    while (i < n1) { arr[k] = L[i]; i++; k++; }
    while (j < n2) { arr[k] = R[j]; j++; k++; }
}

void mergeSort(vector<Estudiante> &arr, int izq, int der) {
    if (izq < der) {
        int medio = izq + (der - izq) / 2;
        mergeSort(arr, izq, medio);
        mergeSort(arr, medio + 1, der);
        merge(arr, izq, medio, der);
    }
}

void ordenarPorPromedio() {
    if (hoja.empty()) return;
    guardarHistorial();
}

```

```

    quickSort(hoja, 0, static_cast<int>(hoja.size() - 1));
    actualizarHash();
    cout << "[OK] Filas ordenadas por Promedio (Descendente) via QuickSort.
";
}

void ordenarPorCodigo() {
    if (hoja.empty()) return;
    guardarHistorial();
    mergeSort(hoja, 0, static_cast<int>(hoja.size() - 1));
    actualizarHash();
    cout << "[OK] Filas ordenadas por Codigo (Ascendente) via MergeSort.
";
}

void deshacer() {
    if (!historial.empty()) {
        hoja = historial.top();
        historial.pop();
        actualizarHash();
        cout << "[UNDO] Estado de celdas restaurado al punto de control anterior.
";
    } else {
        cout << "[INFO] No existen acciones registradas para deshacer.
";
    }
}

void agregarColaAtencion() {
    string codigo;
    cout << "
Ingrese el codigo del estudiante que entra a la cola: ";
    cin >> codigo;

    if (indiceHash.find(codigo) != indiceHash.end()) {
        colaAtencion.push(codigo);
        cout << "[COLA] Estudiante ingresado en la cola FIFO de secretaria.
";
    } else {
        cout << "[ERROR] El codigo no figura registrado en la hoja actual.
";
    }
}

void atenderEstudiante() {
    if (!colaAtencion.empty()) {
        string codigo = colaAtencion.front();
        colaAtencion.pop();
        int pos = indiceHash[codigo];
        cout << "[ATENCION] Despachando requerimiento de: " << hoja[pos].nombre << endl;
    } else {
        cout << "[INFO] Canal de atencion despejado. Cola vacia.
";
    }
}

// Funcion Recursiva Pura para la acumulacion de promedios O(n)
float sumaPromediosRecursiva(size_t i) {
    if (i == hoja.size()) return 0.0f;
    return hoja[i].promedio + sumaPromediosRecursiva(i + 1);
}

void estadisticas() {
    if (hoja.empty()) {
        cout << "[AVISO] Hoja sin datos para el calculo estadistico.
";
        return;
    }
}

```

```

    }
    float suma = sumaPromediosRekursiva(0);
    float promedioGeneral = suma / static_cast<float>(hoja.size());

    cout << "
===== METRICAS ANALITICAS GENERALES =====
";
    cout << " Estudiantes Procesados : " << hoja.size() << "
";
    cout << " Promedio General Coho. : " << fixed << setprecision(2) << promedioGeneral <<
    "
";
    cout << "=====
";
}

void menu() {
    int opcion;
    do {
        cout << "
=====
";
        cout << "          MENU PRINCIPAL - SPREADSHEET
";
        cout << "=====
";
        cout << "1. Registrar Estudiante (Fila)
";
        cout << "2. Mostrar Hoja de Calculo
";
        cout << "3. Buscar Estudiante porCodigo (Hash)
";
        cout << "4. Ordenar por Promedio [QuickSort Desc]
";
        cout << "5. Ordenar por Codigo [MergeSort Asc]
";
        cout << "6. Deshacer Ultima Modificacion [Stack]
";
        cout << "7. Insertar Estudiante a Cola de Atencion
";
        cout << "8. Atender Siguiente Estudiante [FIFO]
";
        cout << "9. Mostrar Estadisticas Consolidadas
";
        cout << "10. Salir de la Aplicacion
";
        cout << "-----
";
        cout << "Seleccione una opcion (1-10): ";

        if (!(cin >> opcion)) {
            cout << "[ALERTA] Entrada invalida.
";
            cin.clear(); cin.ignore(10000, '
'); continue;
        }

        switch (opcion) {
            case 1: registrarEstudiante(); break;
            case 2: mostrarHoja(); break;
            case 3: buscarEstudiante(); break;
            case 4: ordenarPorPromedio(); break;
            case 5: ordenarPorCodigo(); break;
            case 6: deshacer(); break;
            case 7: agregarColaAtencion(); break;
            case 8: atenderEstudiante(); break;
            case 9: estadisticas(); break;

```



```

        case 10: cout << "Liberando memoria interna... Programa cerrado.
"; break;
        default: cout << "[ERROR] Opcion fuera de rango.
";
    }
} while (opcion != 10);
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    menu();
    return 0;
}

```

7. Código Fuente Optimizado en Python

Script estructurado de manera modular utilizando clonaciones profundas (`deepcopy`) para mitigar efectos colaterales de mutabilidad en colecciones anidadas.

```
from collections import deque
from copy import deepcopy

# Estructuras de datos globales
hoja = []
indice_hash = {}
historial = []
cola_atencion = deque()

def calcular_promedio(n1, n2, n3):
    return (n1 + n2 + n3) / 3.0

def actualizar_hash():
    indice_hash.clear()
    for idx, estudiante in enumerate(hoja):
        indice_hash[estudiante["codigo"]] = idx

def guardar_historial():
    # Uso de deepcopy para aislar estados mutables dentro del Stack
    historial.append(deepcopy(hoja))

def registrar_estudiante():
    print("
    --- REGISTRAR NUEVO ESTUDIANTE ---")
    codigo = input("Codigo unico: ").strip()

    if codigo in indice_hash:
        print("[ERROR] Llave primaria duplicada. El codigo ya existe.")
        return

    nombre = input("Apellidos y Nombres: ").strip()
    try:
        nota1 = float(input("Nota 1 (0-20): "))
        nota2 = float(input("Nota 2 (0-20): "))
        nota3 = float(input("Nota 3 (0-20): "))
    except ValueError:
        print("[ERROR] Valores numericos incorrectos en calificaciones.")
        return

    estudiante = {
        "codigo": codigo,
        "nombre": nombre,
        "nota1": nota1,
        "nota2": nota2,
        "nota3": nota3,
        "promedio": round(calcular_promedio(nota1, nota2, nota3), 2)
    }

    guardar_historial()
    hoja.append(estudiante)
    actualizar_hash()
    print("[EXITO] Estudiante registrado e indexado en la estructura asociativa.")

def mostrar_hoja():
    if not hoja:
        print("
    [AVISO] La hoja de calculo no contiene registros activos.")
    return
```

```

    print("
" + "="*70)
    print(f"{'MINI HOJA DE CALCULO ACADEMICA':^70}")
    print("="*70)
    print(f"{'Codigo':<12}{'Nombre Completo':<25}{'N1':<8}{'N2':<8}{'N3':<8}
{'Promedio':<10}")
    print("-"*70)
    for e in hoja:
        print(f"{e['codigo']:<12}{e['nombre']:<25}{e['nota1']:<8.1f}{e['nota2']:<8.1f}
{e['nota3']:<8.1f}{e['promedio']:<10.2f}")
    print("="*70)

def buscar_estudiante():
    codigo = input("
Ingrese codigo a buscar en la estructura asociativa: ").strip()
    if codigo in indice_hash:
        pos = indice_hash[codigo]
        e = hoja[pos]
        print(f"
[ENCONTRADO 0(1)] {e['nombre']} | Promedio Evaluado: {e['promedio']:.2f}")
    else:
        print("[AVISO] El codigo buscado no existe en la base indexada.")

def quicksort(lista):
    if len(lista) <= 1:
        return lista

    pivote = lista[-1]
    # Clasificación de Mayor a Menor (Descendente por Promedio)
    mayores = [x for x in lista[:-1] if x["promedio"] >= pivote["promedio"]]
    menores = [x for x in lista[:-1] if x["promedio"] < pivote["promedio"]]

    return quicksort(mayores) + [pivote] + quicksort(menores)

def merge_sort(lista):
    if len(lista) <= 1:
        return lista

    medio = len(lista) // 2
    izquierda = merge_sort(lista[:medio])
    derecha = merge_sort(lista[medio:])

    return merge(izquierda, derecha)

def merge(izquierda, derecha):
    resultado = []
    i = j = 0

    # Clasificación de Menor a Mayor alfanumérico (Ascendente por Código)
    while i < len(izquierda) and j < len(derecha):
        if izquierda[i]["codigo"] <= derecha[j]["codigo"]:
            resultado.append(izquierda[i])
            i += 1
        else:
            resultado.append(derecha[j])
            j += 1

    resultado.extend(izquierda[i:])
    resultado.extend(derecha[j:])
    return resultado

def ordenar_por_promedio():
    global hoja
    if not hoja: return
    guardar_historial()
    hoja = quicksort(hoja)
    actualizar_hash()

```

```

    print("[OK] Matriz ordenada por Promedio via QuickSort Funcional.")

def ordenar_por_codigo():
    global hoja
    if not hoja: return
    guardar_historial()
    hoja = merge_sort(hoja)
    actualizar_hash()
    print("[OK] Matriz ordenada por Codigo via MergeSort Estable.")

def deshacer():
    global hoja
    if historial:
        hoja = historial.pop()
        actualizar_hash()
        print("[REVERSION] Estado de memoria restaurado a la version previa.")
    else:
        print("[INFO] El historial de modificaciones esta vacio.")

def agregarColaAtencion():
    codigo = input("
Ingrese codigo del estudiante en espera: ").strip()
    if codigo in indice_hash:
        cola_atencion.append(codigo)
        print("[COLA] Estudiante insertado en la cola FIFO institucional.")
    else:
        print("[ERROR] Codigo no registrado en las celdas vigentes.")

def atender_estudiante():
    if cola_atencion:
        codigo = cola_atencion.popleft()
        pos = indice_hash[codigo]
        print(f"[ATENCION] Despachando requerimiento de: {hoja[pos]['nombre']}")
    else:
        print("[INFO] No se registran estudiantes pendientes en la cola.")

def suma_promedios_recursiva(i):
    if i == len(hoja):
        return 0.0
    return hoja[i]["promedio"] + suma_promedios_recursiva(i + 1)

def estadisticas():
    if not hoja:
        print("[AVISO] Datos insuficientes para el analisis.")
        return
    suma = suma_promedios_recursiva(0)
    promedio_general = suma / len(hoja)
    print("
" + "="*40)
    print(f" Estudiantes Registrados : {len(hoja)}")
    print(f" Promedio General Cohorte: {promedio_general:.2f}")
    print("="*40)

def menu():
    while True:
        print("
" + "="*45)
        print("      MENU PRINCIPAL - PYTHON CORE SYSTEMS      ")
        print("="*45)
        print("1. Registrar estudiante (Fila)")
        print("2. Mostrar hoja de calculo")
        print("3. Buscar estudiante por codigo (Hash)")
        print("4. Ordenar por promedio (QuickSort)")
        print("5. Ordenar por codigo (MergeSort)")
        print("6. Deshacer ultima accion (Stack)")
        print("7. Agregar a cola de espera (Queue)")
        print("8. Atender siguiente estudiante (FIFO)")

```

```

print("9. Analitica y Estadisticas generales")
print("10. Salir del programa")
print("-" * 45)

opcion = input("Seleccione una opcion (1-10): ").strip()

if opcion == "1": registrar_estudiante()
elif opcion == "2": mostrar_hoja()
elif opcion == "3": buscar_estudiante()
elif opcion == "4": ordenar_por_promedio()
elif opcion == "5": ordenar_por_codigo()
elif opcion == "6": deshacer()
elif opcion == "7": agregarCola_atencion()
elif opcion == "8": atender_estudiante()
elif opcion == "9": estadisticas()
elif opcion == "10":
    print("Finalizando subprocessos interpretados... Salida.")
    break
else:
    print("[ERROR] Codigo numerico no asignado en el menu.")

if __name__ == "__main__":
    menu()

```

8. Actividades Obligatorias del Estudiante

Actividad 1: Mapeo Arquitectónico del Caso

El estudiante deberá completar y fundamentar teóricamente la elección de cada estructura utilizada:

Función del Sistema	Estructura Usada	Justificación Técnica en Ingeniería de Software
Guardar estudiantes	Lista Dinámica	Asegura localidad de referencia contigua en memoria, ideal para iteraciones rápidas y recorridos de renderizado completo.
Buscar por código	Tabla Hash	Evita el costo lineal de un barrido manual, reduciendo el acceso a un tiempo constante $O(1)$ mediante funciones de dispersión.
Deshacer cambios	Pila (Stack)	Implementa de forma natural la política LIFO, reteniendo cronológicamente los estados inversos para restaurar la memoria.
Atención académica	Cola (Queue)	Modela un flujo equitativo sin prioridad (FIFO), garantizando que el primer elemento en ingresar sea el primero en ser atendido.
Promedio general	Recursividad	Permite expresar reducciones acumulativas compactas utilizando de manera directa el Stack Frame de la arquitectura del procesador.
Ordenar por promedio	QuickSort	Algoritmo de alta eficiencia en el caso promedio con un coste espacial despreciable e in-situ ($O(\log n)$).
Ordenar por código	MergeSort	Garantiza estabilidad algorítmica (no permuta llaves idénticas) y asegura un peor caso acotado de $O(n \log n)$.

Actividad 2: Cuantificación Compleja Asintótica

Complete la siguiente tabla técnica analizando los peores escenarios de ejecución:

Operación Crítica	Estructura / Algoritmo	Complejidad Temporal Peor Caso	Complejidad Espacial Auxiliar
Registrar Estudiante	Lista + Tabla Hash	$O(n)$ (si ocurre Rehashing masivo)	$O(1)$
Mostrar Hoja Completa	Bucle sobre Lista Dinámica	$O(n)$	$O(1)$
Buscar Celda Estudiante	Búsqueda indexada en Hash	$O(n)$ (si el Hash colisiona en un único bucket)	$O(1)$
Ordenar por Rendimiento	Algoritmo QuickSort	$O(n^2)$ (si el pivote coincide con un extremo)	$O(n)$ (en peor caso de recursión)
Ordenar por Identificador	Algoritmo MergeSort	$O(n \log n)$	$O(n)$ (debido a subarreglos de fusión)
Comando Deshacer	Pila Balanceada (LIFO)	$O(1)$ en C++ / $O(n)$ en Python	$O(n)$ por tamaño de instantánea guardada
Encolar Alumno	Operación Enqueue	$O(1)$	$O(1)$
Despachar Alumno	Operación Dequeue	$O(1)$	$O(1)$
Promedio Acumulado	Función Recursiva Lineal	$O(n)$	$O(n)$ (por marcos de activación acumulados)

Actividad 3: Preguntas de Análisis Crítico (Para el Informe de Sustentación)

- ¿Por qué los motores internos de hojas de cálculo como Microsoft Excel o Google Sheets no pueden estructurarse exclusivamente con arreglos unidimensionales o listas enlazadas simples? Fundamente su respuesta considerando accesos indexados bidimensionales.
- Demuestre matemáticamente qué ocurre con el tiempo de búsqueda en una Tabla Hash cuando el factor de carga ($\alpha = n/m$) tiende a infinito y la función de dispersión genera colisiones sistemáticas.
- ¿Qué estrategia de optimización (por ejemplo, el patrón *Memento* o almacenamiento de deltas de cambio) aplicaría sobre la funcionalidad **Deshacer** para evitar el consumo masivo de memoria que implica duplicar el vector completo en cada operación?
- Describa detalladamente la diferencia del manejo de memoria entre C++ (lenguaje que permite control explícito mediante RAII y gestión de punteros) y Python (que depende de un intérprete con conteo de referencias y un Recolector de Basura - *Garbage Collector*) al destruir estados dentro de la pila de historial.

9. Trabajo de Investigación Mandatorio

El estudiante presentará un informe formal con rigor de publicación académica, con una extensión máxima de **6 páginas**, titulado: *"Estructuras de datos aplicadas al diseño de motores de hojas de cálculo y arquitectura de sistemas académicos robustos"*.

El documento deberá regirse bajo la siguiente estructura clásica:

- **Introducción:** El reto del procesamiento de celdas y dependencias en memoria volátil.

- **Marco Teórico:** Análisis matemático del comportamiento asintótico de listas dinámicas, pilas estructuradas, colas operativas y mapas asociativos.
- **Arquitectura del Sistema Integrado:** Explicación de cómo interactúa el índice Hash con el almacenamiento secuencial de registros.
- **Comparativa Transversal de Paradigmas:** Evaluación del rendimiento y tipado entre C++ y Python.
- **Conclusiones y Referencias:** Deducciones basadas en métricas e inclusión de literatura indexada bajo normativa APA 7.

10. Entregables del Estudiante

1. **Código Fuente Compilable:** Archivo `.cpp` estructurado y libre de fugas de memoria y script `.py` verificado y modularizado.
2. **Informe Técnico:** Documento final en formato Word o PDF que contenga el desarrollo analítico y la resolución completa de los cuestionarios.
3. **Evidencia de Ejecución:** Capturas de pantalla ordenadas cronológicamente demostrando la consistencia de los datos ante ordenamientos y operaciones de deshacer.
4. **Defensa en Video:** Clip audiovisual explicativo (máximo 10 minutos) analizando línea por línea el flujo del programa y el comportamiento de la memoria interna.

11. Rúbrica y Criterios de Evaluación

Dimensión Evaluativa	Puntaje Máximo	Indicador de Logro de Excelencia
Diseño Estructural del Caso	2 Puntos	Modelado preciso de la lógica del negocio sin mezclar la interfaz con el procesamiento.
Implementación C++ Robusta	3 Puntos	Uso óptimo de tipos de datos, referencias y flujos rápidos libres de advertencias del compilador.
Implementación Python Limpia	3 Puntos	Uso correcto de modismos idiomáticos (Pythonic code) y aislamiento de mutabilidad mediante copias seguras.
Sincronización Híbrida de Estructuras	3 Puntos	Garantía de consistencia absoluta entre la Lista Dinámica y el índice Hash tras operaciones de mutación o reordenamiento.
Algoritmos Avanzados y Estabilidad	2 Puntos	Correcta modularización de QuickSort y MergeSort respetando los criterios de ordenación solicitados.
Control de Recursividad	1 Punto	Definición infalible del caso base evitando ciclos infinitos o desbordamientos de pila.
Análisis Estadístico Asintótico	2 Puntos	Demostración analítica rigurosa de las complejidades temporales y espaciales en las tablas.
Calidad del Informe Técnico	2 Puntos	Sustentación clara, redacción formal en ingeniería y cumplimiento estricto de citas bibliográficas.
Sustentación y Defensa Analítica	2 Puntos	Fluidez verbal, dominio conceptual absoluto de las estructuras de datos y claridad explicativa en el video.
Total Máximo	20 Puntos	Nota vigesimal máxima aprobatoria según estándares institucionales.

12. Referencias Bibliográficas Normadas (APA 7)

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press. <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>
- cppreference.com. (2026). `std::unordered_map`. https://en.cppreference.com/w/cpp/container/unordered_map
- cppreference.com. (2026). `std::queue`. <https://en.cppreference.com/w/cpp/container/queue>
- Morin, P. (2013). *Open data structures: An introduction*. Athabasca University Press. <https://www.aupress.ca/books/120226-open-data-structures/>
- Python Software Foundation. (2026). *collections* — *Container datatypes*. Python Documentation. <https://docs.python.org/3/library/collections.html>
- Python Software Foundation. (2026). *Data structures*. Python Documentation. <https://docs.python.org/3/tutorial/datastructures.html>
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.
- Weiss, M. A. (2014). *Data structures and algorithm analysis in C++* (4th ed.). Pearson.

Recomendación institucional de cierre: Esta guía práctica no tiene como único fin la codificación mecánica de una solución básica. Su verdadero propósito radica en la comprensión profunda de que, detrás de interfaces de usuario sencillas o celdas interactivas, operan estructuras de datos complejas que definen la eficiencia, viabilidad y escalabilidad de los sistemas de software moderno.